
LexAI Legal Tech Platform

ENTERPRISE PRODUCTION BLUEPRINT & SECURITY SPECIFICATION

Target Stack: React 19, Vite 8, Django 6, PostgreSQL

Security Standard: OAuth2, JWT Session, SSL Pinning, CSP

Author: Enterprise Systems Architect

Document Version: v2.0 (Production Hardened)

01. Browser & App Cache Security Architecture

To ensure corporate compliance and user data confidentiality, sensitive legal records, client files, and authorization credentials must be heavily guarded against local storage leaks, Cross-Site Scripting (XSS), and Man-in-the-Middle (MitM) interceptions.

A. Web Browser Caching & Storage Guidelines

1. JWT Storage & Session Strategy: Access tokens should be kept purely in React's component state (in-memory). Refresh tokens must be stored in secure HttpOnly cookies set with the following flags:

- **HttpOnly:** Disallows JavaScript reading access, fully preventing token theft via XSS injection.
- **Secure:** Forces cookie transmission only over encrypted TLS/HTTPS connections.
- **SameSite=Strict:** Mitigates Cross-Site Request Forgery (CSRF) attempts by restricting cookie transmission to top-level site navigation.

2. HTTP Cache Control Headers: For all authenticated API endpoints returning sensitive data (e.g., case info, AI chat transcripts), the Django backend must return strict headers to prevent intermediaries and local browsers from storing the response in disk cache:

```
Cache-Control: no-store, no-cache, must-revalidate, max-age=0
Pragma: no-cache
Expires: 0
```

3. Content Security Policy (CSP): Implement strict CSP headers on the server to prevent malicious script execution, unauthorized frames, and clickjacking:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; connect-src 'self'
https://api.indiankanoon.org; frame-ancestors 'none';
```

B. Native Mobile Application Security Guidelines

1. Keychain & Keystore Storage: Mobile applications must never store credentials or private user data in plaintext formats (e.g., AsyncStorage or SharedPreferences). Instead, utilize hardware-backed storage:

- **iOS Keychain:** Hardware-level encryption for cryptographic keys and secrets.
- **Android Keystore System:** Secure cryptographic operations inside Trusted Execution Environments (TEE) or StrongBox keymaster.

2. SSL/TLS Certificate Pinning: To prevent MitM traffic interception, hardcode and validate the backend certificate's public key hash inside the mobile client wrapper. This prevents the app from accepting proxy-injected certificates.

3. Local SQLite Encryption: If caching case documents locally for offline viewing, the SQLite database file must be encrypted using SQLCipher, with the decryption key derived dynamically from the device's hardware keystore.

02. Company-Level Scale & Infrastructure Masterplan

To elevate LexAI from a prototype to a company-level production ecosystem, strict patterns for reliability, scalability, and security must be established.

A. Enterprise CI/CD Pipeline & Environments

Maintain three distinct environment levels: Development, Staging (matching prod spec), and Production. A typical CI/CD flow utilizing GitHub Actions and automated triggers executes:

1. Code linting and formatting validation (eslint, black).
2. Automated unit and integration testing execution (pytest, Jest).
3. Staging deploy with Cypress E2E integration test suites.
4. Production zero-downtime rolling release deploy with automatic rollback triggers.

B. Database Scalability & Connection Optimization

Production database requirements dictate moving away from SQLite. Establish PostgreSQL in a high-availability cluster:

- **Connection Pooling:** Implement PgBouncer as a middleware proxy between Django instances and PostgreSQL to reuse active connections and prevent overhead scaling bottlenecks.
- **Read/Write Replication:** Direct write operations to a primary instance and scale search queries across read-only replica servers.
- **Strategic Indexing:** Create composite indexes on high-frequency query fields (e.g., user_email, case_status, article_number).

C. API Rate Limiting & Edge Shielding

Implement Cloudflare or AWS CloudFront at the DNS edge layer. This protects the backend API against DDoS attacks, configures Web Application Firewall (WAF) rule sets, and performs static asset caching close to regional clients. Additionally, configure Django-based rate limits (e.g., 60 requests/minute per IP, 20 queries/day for legal advice) to prevent script-based scraping.

D. Observability, Logging, and Performance Audits

- **Error Tracking:** Integrate Sentry.io in both Django and React to capture runtime exceptions, user session context, and trace frontend-backend transaction errors.
- **System Metrics:** Run Prometheus exporters to capture RAM, CPU, database load, and view API latency performance dashboards in Grafana.

03. Mobile vs. Web UI/UX Adaptive Layout Reference

To achieve top-tier professional usability, the layout dynamically adapts to different viewports, shifting grid sizes and navigational paths to optimize space.

Component / Viewport	Mobile (<768px)	Tablet (768px - 1024px)	Desktop (>1024px)
Main Navigation	Bottom navigation bar with icons	Collapsible slim sidebar (60px, icons only)	Expanded permanent sidebar (220px, labels)
Dashboards	Single column vertical stacking	2-column grid layout	3-column horizontal grid alignment
AI Chat Panel	Full-screen focus, sliding sidebar overlay	Split-pane with sliding history list	Split workspace: history panel fixed on left

By enforcing this combined standard of caching security, cloud scalability, and layout responsiveness, LexAI transitions from a local prototype to a robust, enterprise-grade, company-level platform.